第12章 深度学习扩展

课程：人工智能与地理数据挖掘

教师：孙伟

邮箱：sunwei29@mail.sysu.edu.cn

单位：中山大学 地理科学与规划学院

日期：2026年6月

第12章 深度学习扩展 - 提纲

12.1 深度学习内容扩展概述

12.2 从 RNN/LSTM/CNN 到 Transformer

12.3 Transformer 的基本原理和结构

12.4 MATLAB 深度学习工具箱的内置支持

12.5 GitHub 上已转化成 MATLAB 的深度学习模型

12.6 通过 ONNX 导入深度学习模型

12.7 小结与思考题

12.1 · 深度学习内容扩展概述

- 回顾课程已有的三种经典网络：RNN、LSTM、CNN
- 为什么要引入 Transformer：并行能力与长程依赖
- Transformer 的基本原理：自注意力、多头注意力、位置编码
- 编码器-解码器结构与"注意力就是你所需要"
- MATLAB 深度学习工具箱对 Transformer 的内置支持（R2023a+）
- GitHub 上 MathWorks 官方转化的预训练模型库（BERT / FinBERT / GPT-2）
- 通过 ONNX 导入更广泛的 PyTorch / TensorFlow 模型

学习目标

> 在已有 RNN、LSTM 和 CNN 基础上，补上通往大语言模型的那块基石
> 完成本章后，你将能够：

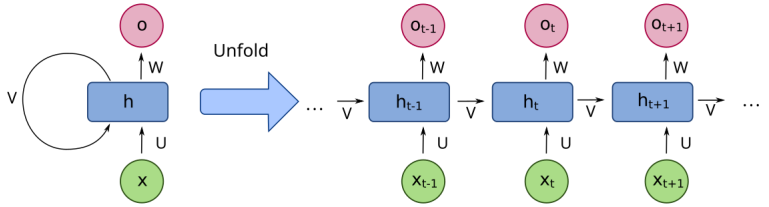
- **说清** Transformer 相比 RNN/LSTM/CNN 的优势与原理
- **手写** 自注意力公式，解释多头注意力与位置编码的作用
- **使用** MATLAB 内置注意力层搭建简单的 Transformer 网络
- **直接调用** 官方 transformer-models 仓库中的 BERT、FinBERT、GPT-2
- **通过 ONNX** 把 PyTorch/TF 训练的模型导入 MATLAB 使用

已有深度学习基础

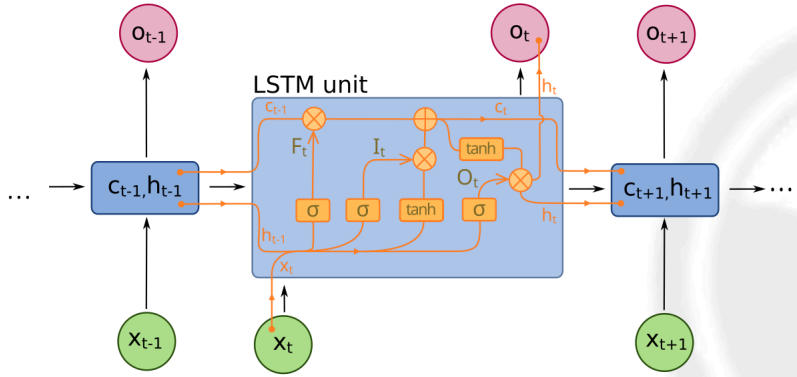
在本课程已有的深度学习内容基础上，回顾三种经典网络：

- RNN（循环神经网络）：处理序列数据，具有时间步依赖
- LSTM（长短期记忆网络）：通过门控机制缓解 RNN 的长程依赖问题
- CNN（卷积神经网络）：通过卷积核提取局部空间特征，擅长图像与序列模式

RNN 按时间步展开：顺序依赖，难并行



LSTM 单元：门控机制缓解长程依赖



本次扩展的两个方向

在 RNN、LSTM、CNN 基础上，本部分新增两方面内容：

- **Transformer 模型的基本原理和结构**
  - 自注意力机制带来的并行能力与长程依赖建模能力
  - 编码器-解码器架构，已成为大语言模型的基石
- **GitHub 上已经转化成 MATLAB 的深度学习模型**
  - 介绍 MathWorks 官方维护的预训练模型库
  - 学生能够直接在 MATLAB 中使用、理解和微调这些模型

一句话：Transformer 是通往大语言模型的**必经之路**，MATLAB 模型库让你**不必从零搭起**

三类网络的特征对比

网络	计算方式	并行性	长程依赖	典型应用
RNN/LSTM	按时间步循环	差（顺序依赖）	较弱	语音、时间序列
CNN	局部卷积滑动	好	受限于感受野	图像、文本分类
Transformer	**全局自注意力**	**好（序列并行）**	**强**	**机器翻译、LLM**

Transformer 同时拿到 CNN 的"可并行"与超越 RNN 的"长记忆"——这是它能做"大"的根本原因

第12章 深度学习扩展

12.1 深度学习内容扩展概述

12.2 从 RNN/LSTM/CNN 到 Transformer

12.3 Transformer 的基本原理和结构

12.4 MATLAB 深度学习工具箱的内置支持

12.5 GitHub 上已转化成 MATLAB 的深度学习模型

12.6 通过 ONNX 导入深度学习模型

12.7 小结与思考题

为什么要引入 Transformer

现有模型的缺陷

- RNN 的**顺序计算**阻碍并行训练，长序列梯度传播困难
- CNN 的**感受野有限**，捕获远距离依赖需要堆叠多层

Transofrmer 的优势

- Transformer 通过**自注意力**，让序列中任意两个位置直接交互：
  - 最大路径长度为 O(1)，便于学习远距离依赖
  - 没有循环结构，序列内单词可**同时训练**，效率更高
  - 通过**位置编码**注入顺序信息

核心思想：注意力就是所有你需要的

2017 年 Vaswani 等人在论文《Attention is All You Need》中提出 Transformer：

- 一种基于**自注意力机制**的编码器-解码器（seq2seq）神经网络
- 用于机器翻译任务，达到当时的 **SOTA** 效果
- 完全摒弃循环结构，**仅用注意力机制**建模序列依赖

> Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. NeurIPS 2017: 5998-6008

2017 年的这篇论文，是今日所有主流大语言模型的**共同起点**

自注意力机制

给定输入序列的查询 Q、键 K、值 V，缩放点积注意力为：

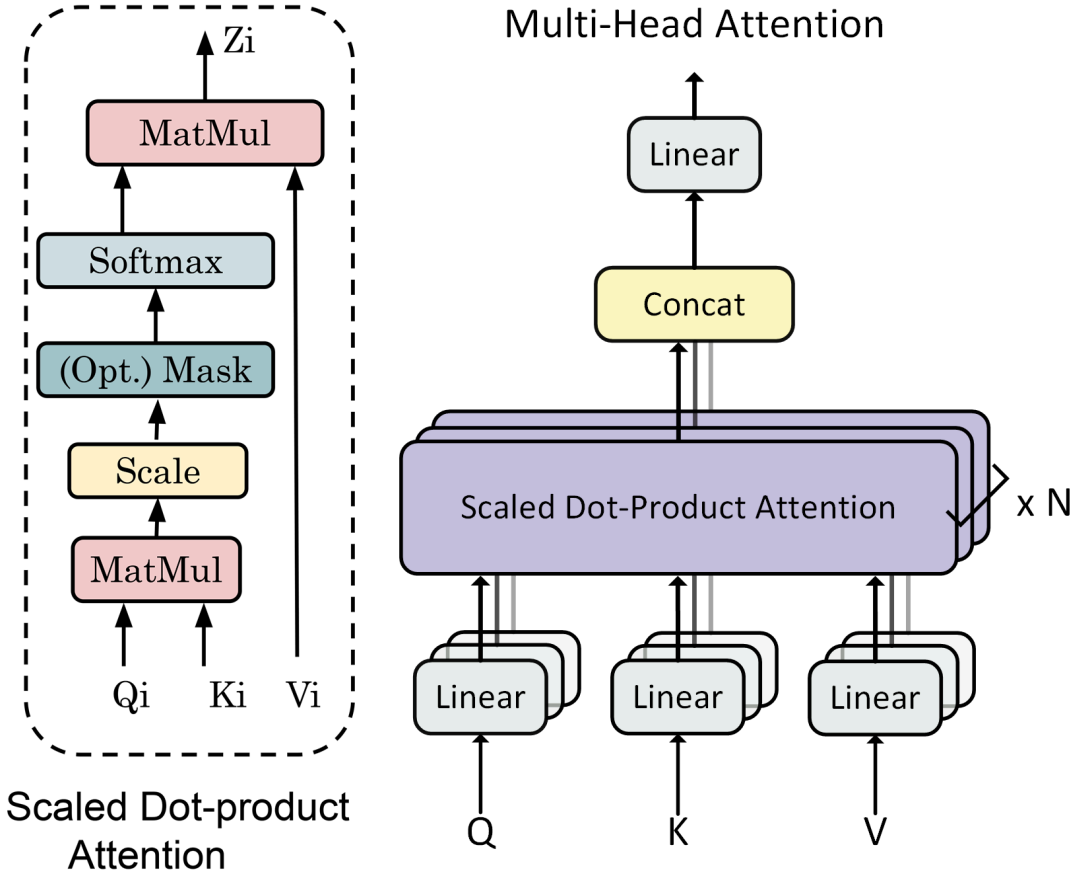
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- 每个查询与所有键计算相似度，经 softmax 得到**注意力权重**
- 用注意力权重对值加权求和，得到输出
- 同一组词元同时充当查询、键和值，故称**自注意力**

多头注意力

- 用 h 组独立学习的线性投影变换 Q、K、V
- h 组变换并行送入注意力汇聚，捕获**不同子空间**的依赖
- 拼接 h 个头的输出，再经一次线性投影得到最终输出
- 不同头可关注输入不同部分，表达比简单加权平均更复杂的函数

注意力举例：缩放点积注意力（左）与多头注意力（右）



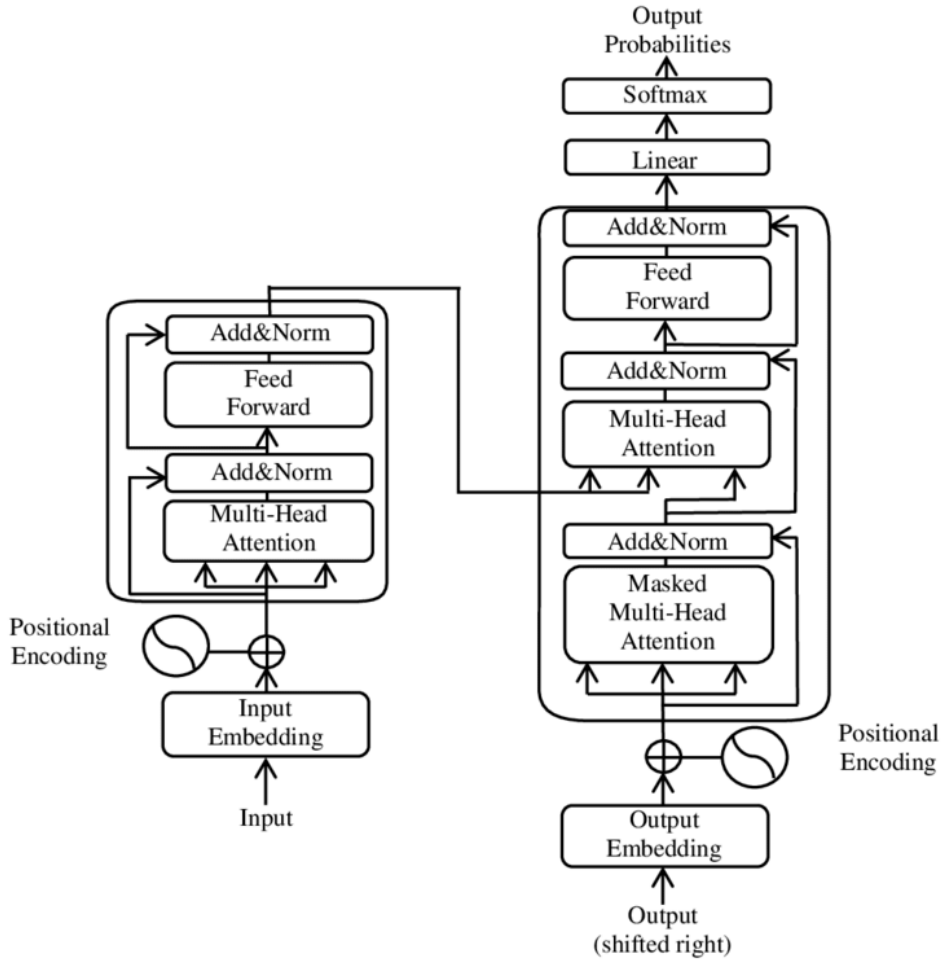
位置编码

- 自注意力**并行计算，本身不含顺序信息**
- 通过在输入嵌入中添加**位置编码**注入绝对/相对位置信息
- 经典公式（正弦/余弦）：
  - $$PE(pos, 2i) = sin(pos/10000^{(2i/d_{model})})$$
  - $$PE(pos, 2i+1) = cos(pos/10000^{(2i/d_{model})})$$
- sin/cos 的周期性与相位特性可捕捉**相对位置**信息

没有位置编码，Transformer 就是一堆"无序词袋"——顺序信息全靠它

Transformer 宏观结构

- **编码器-解码器架构**
- 编码器输入：模型总输入 X（1 到 T）
- 解码器输入：编码器输出隐藏层信息 + 已生成的输出序列
- 经典配置：**6 个编码器 + 6 个解码器**堆叠



编码器结构

每个编码器层包含两个子层：

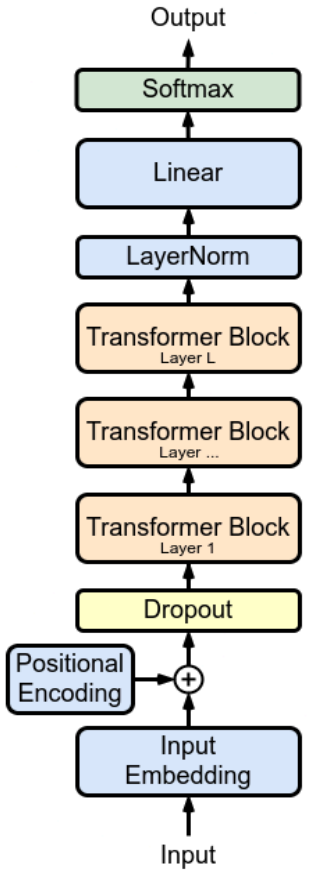
- **多头注意力机制**（自注意力）
- **前馈神经网络**（Feed Forward）

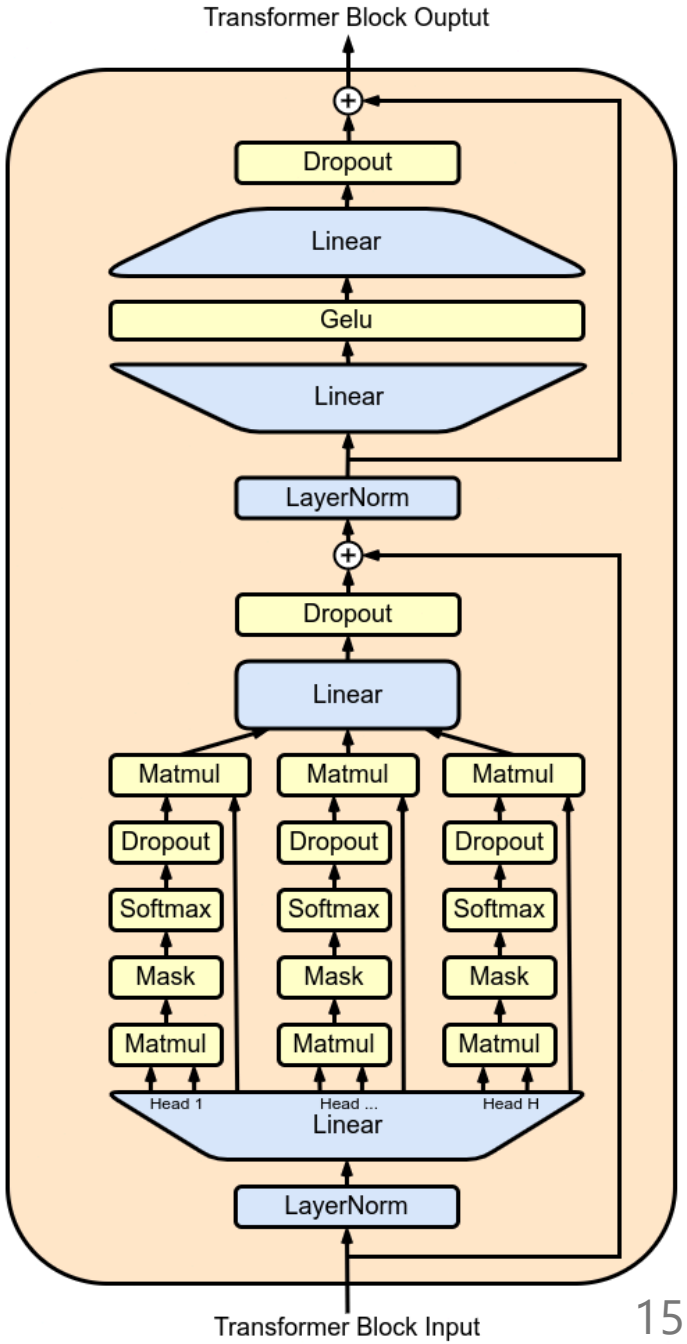
每个子层采用残差连接（Add）与层归一化（LayerNorm）：

$$\text{sublayer_output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

解码器多一层**掩蔽多头注意力**，防止看到未来位置信息。

每个子层 = "注意力/前馈 + 残差 + 归一化"





Transformer 的深远影响

年份	模型	结构	意义
2017	Transformer	编码器-解码器	提出自注意力编解码架构
2018	BERT	仅编码器	预训练+微调，11 项 NLP 任务 SOTA
2018	GPT	仅解码器	生成式预训练
2018-	GPT-2/3/4、ChatGPT	解码器-only	大语言模型时代的开始

第12章 深度学习扩展

12.1 深度学习内容扩展概述

12.2 从 RNN/LSTM/CNN 到 Transformer

12.3 Transformer 的基本原理和结构

12.4 MATLAB 深度学习工具箱的内置支持

12.5 GitHub 上已转化成 MATLAB 的深度学习模型

12.6 通过 ONNX 导入深度学习模型

12.7 小结与思考题

12.4 · MATLAB 深度学习工具箱的内置支持

- R2023a/R2023b 起内置 Transformer 相关层：selfAttentionLayer、positionEmbeddingLayer 等
- 组合内置层即可搭建一维序列的迷你 Transformer
- Deep Network Designer 支持可视化拖拽搭建
- R2025a 起可导入 Hugging Face Transformer 层

内置 Transformer 相关层（R2023a/R2023b 起）

深度学习工具箱内置了可直接搭建 Transformer 的层：

- **selfAttentionLayer**：自注意力层
- **attentionLayer**：通用注意力层
- **positionEmbeddingLayer**：位置嵌入层
- **indexing1dLayer**：一维索引层
- **patchEmbeddingLayer**（R2023b）：补丁嵌入层，用于视觉 Transformer
- **embeddingConcatenationLayer**：嵌入拼接层

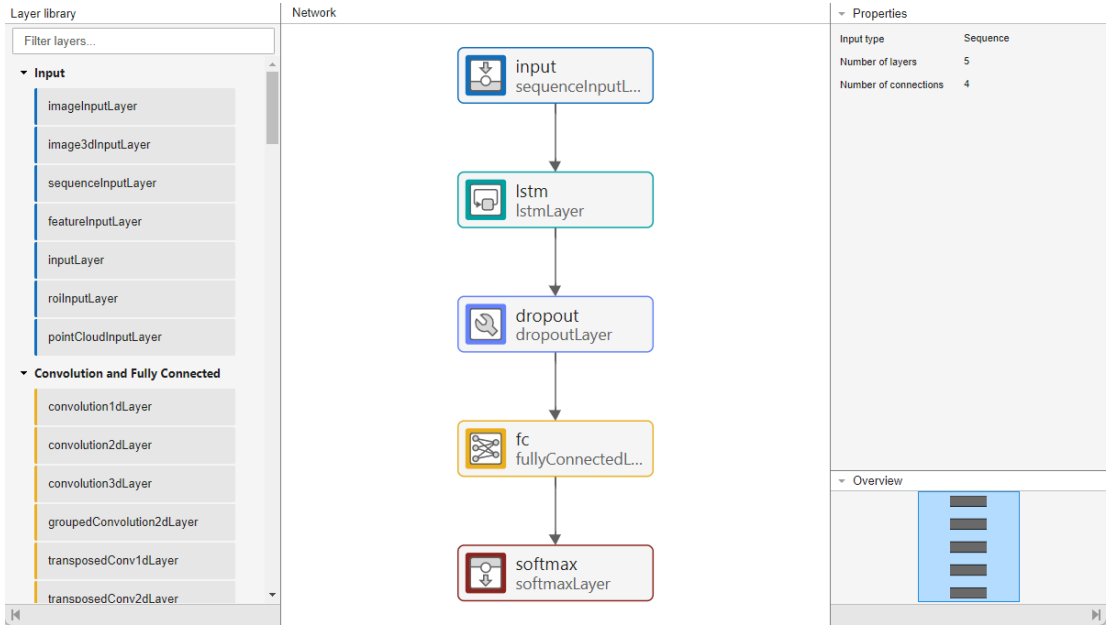
不用写一行 Python，纯 MATLAB 即可**从零搭建 Transformer**

组合示例：一维序列的迷你 Transformer

时间序列预测 / 文本分类等任务，可以直接用内置层组合：

```
numHeads = 4; dim = 64;
layers = [
    sequenceInputLayer(1,"Normalization","zscore")
    positionEmbeddingLayer(128,dim)      % 位置嵌入：128 个位置
    embeddingConcatenationLayer          % 拼接序列嵌入与位置嵌入
    selfAttentionLayer(numHeads,dim)     % 4 头自注意力
    fullyConnectedLayer(1)
    regressionLayer];
net = trainNetwork(XTrain, YTrain, layers, options);
```

在 Deep Network Designer 中搭建 Transformer



- 上述层可在 Deep Network Designer 中**可视化拖拽**组合
- 支持自定义**一维序列数据**的 Transformer 网络
- 适用于时间序列预测、文本分类等任务

图片来源：MathWorks 官方文档《Get Started with Deep Network Designer》

R2025a 起：导入 Hugging Face Transformer 层

- 通过 **importNetworkFromPyTorch**（R2025a 引入）导入 PyTorch 模型
- 可识别并映射 Hugging Face 中的 Transformer 层：
  - BertAttention、RobertaSelfAttention
  - DistilBERT MultiheadSelfAttention 等
- 配合 Deep Network Designer 查看、编辑导入的网络

Hugging Face 上的海量模型，从此有了**直通 MATLAB** 的入口

第12章 深度学习扩展

12.1 深度学习内容扩展概述

12.2 从 RNN/LSTM/CNN 到 Transformer

12.3 Transformer 的基本原理和结构

12.4 MATLAB 深度学习工具箱的内置支持

12.5 GitHub 上已转化成 MATLAB 的深度学习模型

12.6 通过 ONNX 导入深度学习模型

12.7 小结与思考题

12.5 · GitHub 上已转化成 MATLAB 的深度学习模型

MathWorks 官方 transformer-models 仓库：经典模型的纯 MATLAB 实现

三大模型家族：BERT、FinBERT、GPT-2

支持 "Open in MATLAB Online"，开箱即用

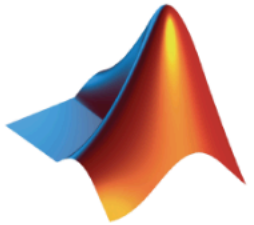
MathWorks 官方 Transformer 模型库

MathWorks 在 GitHub 维护了 **Transformer Models for MATLAB** 仓库，把经典预训练 Transformer 模型用**纯 MATLAB 代码**实现：

- GitHub：**matlab-deep-learning/transformer-models**
- 也可在 File Exchange 或 Add-On Explorer 搜索安装
- 支持 **"Open in MATLAB Online"**，无需本地配置即可在线运行

matlab-deep-learning/
transformer-models

Deep Learning Transformer models in MATLAB



6 13 243 64



仓库包含的模型

该仓库目前包含三个核心模型家族：

模型	结构	典型任务
BERT	双向编码器	文本理解与分类
FinBERT	金融领域 BERT	金融情感分析
GPT-2	解码器	文本生成与摘要

注：仓库不含 T5 等编码器-解码器文本到文本模型；此类模型需通过 ONNX 导入或 Python 接口调用。

BERT 在 MATLAB 中的使用

- `mdl = bert`：加载预训练 BERT（自动下载权重）
- 指定变体：`mdl = bert("Model",modelName)`
  - base、**multilingual-cased（多语言，含中文）**、medium、small、mini、tiny、japanese-base
- 推理：`Z = mdl.model(X, parameters)`，输出 token 级 BERT 嵌入
- 配套示例：ClassifyTextDataUsingBERT.m、FineTuneBERT.m、PredictMaskedTokensUsingBERT.m

```
mdl = bert("Model","multilingual-cased");    % 多语言模型，支持中文
Z = mdl.model(X, mdl.Parameters);            % X 为 token 索引序列，Z 为 token 级嵌入
```

FinBERT 在 MATLAB 中的使用

- 专门面向**金融文本**的 BERT 变体
- `finbert.sentimentModel(X, parameters)` 输出情感分类：
  - 类别：**positive / neutral / negative**
  - 分值：[-1, 1] 区间
- 配套示例：SentimentAnalysisWithFinBERT.m、PredictMaskedTokensUsingFinBERT.m

金融新闻、财报、舆情文本的情感分析入门首选——**开箱即用**

GPT-2 在 MATLAB 中的使用

- `mdl = gpt2`：加载预训练 GPT-2 模型
- 文本摘要：`summary = generateSummary(mdl, text)`
  - 自动在输入后追加 **"TL;DR"**，利用 GPT-2 生成式摘要
  - 参数：MaxSummaryLength（默认 50）、TopK（默认 2）、Temperature（默认 1）、StopCharacter（默认 "."）
- 仓库含 **+sampling 包**，支持 top-k、temperature 等采样策略
- 配套示例：SummarizeTextUsingTransformersExample.m

仓库中的 Transformer 自定义层

仓库 +transformer/+layer 子包提供可在自定义网络中复用的层：

- 多头注意力层
- 位置嵌入与前馈层
- 层归一化与残差结构

读源码学原理：每个 Transformer 组件的 MATLAB 实现都摆在眼前，适合教学精读

第12章 深度学习扩展

12.1 深度学习内容扩展概述

12.2 从 RNN/LSTM/CNN 到 Transformer

12.3 Transformer 的基本原理和结构

12.4 MATLAB 深度学习工具箱的内置支持

12.5 GitHub 上已转化成 MATLAB 的深度学习模型

12.6 通过 ONNX 导入深度学习模型

12.7 小结与思考题

12.6 · 通过 ONNX 导入深度学习模型

- ONNX：PyTorch/TensorFlow 模型的跨框架"通用语"
- importNetworkFromONNX：把 .onnx 模型导入 MATLAB 推理与微调
- 覆盖官方仓库之外的更广泛模型

为什么需要 ONNX

- 除官方仓库的 BERT/FinBERT/GPT-2 外，大量模型以 **ONNX 格式**分发
- **ONNX**（Open Neural Network Exchange）是跨框架的模型交换格式
- 可将 PyTorch、TensorFlow 训练的模型**导入 MATLAB** 推理与微调

PyTorch / TensorFlow	**ONNX 文件（.onnx）**	MATLAB
（训练社区的主力）	（跨框架"通用语"）	（推理 / 微调 / 部署）

MATLAB 导入 ONNX 的函数

- **importNetworkFromONNX**（R2023b 引入，推荐）：导入为 dlnetwork 对象，支持代码生成
- importONNXNetwork（旧函数，逐步弃用）：导入为 DAGNetwork 或 dlnetwork
- 前提：安装 **"Deep Learning Toolbox Converter for ONNX Model Format"** 支持包
- 遇到无法映射的算子时，自动生成**自定义层**（保存于 +modelfile 命名空间）
- 导出方向：**exportONNXNetwork**，可将 MATLAB 网络导出为 ONNX

ONNX 导入流程示例

1. 在 PyTorch/TF 中训练或获取模型，导出为 .onnx 文件
2. MATLAB 中安装 ONNX 转换支持包
3. `net = importNetworkFromONNX("model.onnx")` 导入为 dlnetwork
4. 用 `predict(net, X)` 或 `trainnet` 进行推理/微调
5. 必要时在 Deep Network Designer 中检查并修改网络结构

```
net = importNetworkFromONNX("gpt2.onnx", OutputNames="logits");
scores = predict(net, tokenIndices);   % 推理
```

MATLAB 深度学习模型中心

MathWorks 汇总了官方可用的预训练 Transformer 模型：

任务类型	推荐模型	来源
文本理解 / 分类	BERT	transformer-models 仓库
金融情感分析	FinBERT	transformer-models 仓库
文本生成 / 摘要	GPT-2	transformer-models 仓库
句子/文档嵌入	all-MiniLM-L6-v2 / L12-v2	文档嵌入支持包
图像任务	视觉 Transformer	Computer Vision Toolbox Model

选型：理解分类找 BERT、金融情感用 FinBERT、生成摘要用 GPT-2、语义检索用 MiniLM、图像任务找 ViT

第12章 深度学习扩展

12.1 深度学习内容扩展概述

12.2 从 RNN/LSTM/CNN 到 Transformer

12.3 Transformer 的基本原理和结构

12.4 MATLAB 深度学习工具箱的内置支持

12.5 GitHub 上已转化成 MATLAB 的深度学习模型

12.6 通过 ONNX 导入深度学习模型

12.7 小结与思考题

小结

- 在 RNN/LSTM/CNN 基础上，**Transformer 通过自注意力**实现并行计算与长程依赖建模，成为大语言模型的基石
- MATLAB 深度学习工具箱自 **R2023a/b** 起内置 selfAttentionLayer、positionEmbeddingLayer 等层，可直接搭建 Transformer
- MathWorks 官方 **transformer-models 仓库**提供 BERT、FinBERT、GPT-2 的纯 MATLAB 实现，配套分类、微调、摘要等示例
- 借助 **importNetworkFromONNX**，可导入更广泛的 PyTorch/TF 模型
- 学生可直接在 **MATLAB Online** 中运行示例，理解并改造这些模型

思考题

- 相比 RNN/LSTM，Transformer 为什么能更好地处理长程依赖？
- MATLAB 内置的 selfAttentionLayer 与 transformer-models 仓库中的实现，分别适合什么场景？
- 何时应选择 ONNX 导入而非直接使用官方仓库的模型？
- 如何在 MATLAB 中用 BERT 完成一个中文文本分类任务？

参考文献

- Vaswani A, et al. Attention is All You Need. NeurIPS 2017
- MathWorks. Transformer Models for MATLAB. https://github.com/matlab-deep-learning/transformer-models
- MathWorks. importNetworkFromONNX 文档. https://www.mathworks.com/help/deeplearning/ref/importnetworkfromonnx.html
- MathWorks. 深度学习工具箱预训练网络. https://www.mathworks.com/help/deeplearning/ug/pretrained-convolutional-neural-networks.html

谢谢

延伸阅读

- Vaswani et al., *Attention Is All You Need*, NeurIPS 2017
- MathWorks transformer-models 仓库示例（BERT / FinBERT / GPT-2）